Mapeo Hibernate @ManyToMany

Desarrollo Entorno Servidor

@ManyToMany

Las relaciones N:M son las relaciones más difíciles de gestionar

```
CREATE TABLE PROFESOR
(
   ID_PROFESOR BIGINT NOT NULL AUTO_INCREMENT,
   NOMBRE VARCHAR (40) NOT NULL,
   NICKNAME VARCHAR (40) NOT NULL,
   APELLIDOS VARCHAR (40) NOT NULL,
   PRIMARY KEY (ID_PROFESOR)
);
```

```
CREATE TABLE MODULO
(
   ID_MODULO BIGINT NOT NULL AUTO_INCREMENT,
   NOMBRE_MODULO VARCHAR (40) NOT NULL UNIQUE,
   PRIMARY KEY (ID_MODULO)
);
```

Para establecer una relación N:M necesitamos al menos 3 tablas, ya que necesitaremos una tabla intermedia, en la que contará con las claves foráneas de las dos entidades.

```
CREATE TABLE PROFESOR_MODULO
(
   ID_PROFESOR BIGINT NOT NULL,
   ID_MODULO BIGINT NOT NULL,
   PRIMARY KEY
   (
      ID_PROFESOR,
      ID_MODULO
   ),
   CONSTRAINT FK_PROFESOR_MODULO_1 FOREIGN KEY (ID_PROFESOR) REFERENCES PROFESOR
(ID_PROFESOR) ON DELETE CASCADE,
   CONSTRAINT FK_PROFESOR_MODULO_2 FOREIGN KEY (ID_MODULO) REFERENCES MODULO
(ID_MODULO)
);
```

@ManyToMany

En las relaciones **@ManyToMany** las dos entidades son entidades padre. Ya que los dos son padre los dos , los dos partes de la relación pueden ser navegadas y ninguna de ellas mantendrá claves foráneas.

La tabla intermedia será aquella que mantenga las claves foráneas y juegue el rol como hija.

Profesor: Entidad propietaria

```
public class Profesor implements Serializable {

private Long idProfesor;

private Set<Modulo> modulos = new HashSet<>();
```

Profesor: Entidad propietaria

1. Establecemos las anotaciones básicas.

```
@Entity
@Table(name = "PROFESOR")
public class Profesor implements Serializable {


@Id
@GeneratedValue(strategy= GenerationType.IDENTITY)
@Column(name = "ID_PROFESOR")
private Long idProfesor;
```

Profesor: Entidad propietaria

1. Establecemos las anotaciones básicas.
2. Como un Profesor se relaciona con varios Módulos establecemos una variable módulo de tipo **Set**.

```
@Entity
@Table(name = "PROFESOR")
public class Profesor implements Serializable {


@Id
@GeneratedValue(strategy= GenerationType.IDENTITY)
@Column(name = "ID_PROFESOR")
private Long idProfesor;





private Set<Modulo> modulos = new HashSet<>();
```

Profesor: Entidad propietaria

1. Establecemos las anotaciones básicas.
2. Como un Profesor se relaciona con varios Módulos establecemos una variable módulo de tipo **Set**.
3. Anotamos los módulos con las anotación **@ManyToMany**

```
@Entity
@Table(name = "PROFESOR")
public class Profesor implements Serializable {


@Id
@GeneratedValue(strategy= GenerationType.IDENTITY)
@Column(name = "ID_PROFESOR")
private Long idProfesor;

@ManyToMany




private Set<Modulo> modulos = new HashSet<>();
```

Profesor: Entidad propietaria

1. Establecemos las anotaciones básicas.
2. Como un Profesor se relaciona con varios Módulos establecemos una variable módulo de tipo **Set**.
3. Anotamos los módulos con las anotación **@ManyToMany**
4. Indicamos **CascadeType Persist** y **Merge** para persistir las creaciones y guardar cambios de padres a hijos.

```
@Entity
@Table(name = "PROFESOR")
public class Profesor implements Serializable {


@Id
@GeneratedValue(strategy= GenerationType.IDENTITY)
@Column(name = "ID_PROFESOR")
private Long idProfesor;

@ManyToMany(cascade = {CascadeType.PERSIST,
                       CascadeType.MERGE})



private Set<Modulo> modulos = new HashSet<>();
```

Profesor: Entidad propietaria

1. Establecemos las anotaciones básicas.
2. Como un Profesor se relaciona con varios Módulos establecemos una variable módulo de tipo **Set**.
3. Anotamos los módulos con las anotación **@ManyToMany**
4. Indicamos **CascadeType Persist** y **Merge** para persistir las creaciones y guardar cambios de padres a hijos.
5. Anotación **@JoinTable** para establecer la tabla intermedia

```
@Entity
@Table(name = "PROFESOR")
public class Profesor implements Serializable {


@Id
@GeneratedValue(strategy= GenerationType.IDENTITY)
@Column(name = "ID_PROFESOR")
private Long idProfesor;

@ManyToMany(cascade = {CascadeType.PERSIST,
                       CascadeType.MERGE})
@JoinTable(name = "PROFESOR_MODULO“)


private Set<Modulo> modulos = new HashSet<>();
```

Profesor: Entidad propietaria

1. Establecemos las anotaciones básicas.
2. Como un Profesor se relaciona con varios Módulos establecemos una variable módulo de tipo **Set**.
3. Anotamos los módulos con las anotación **@ManyToMany**
4. Indicamos **CascadeType Persist** y **Merge** para persistir las creaciones y guardar cambios de padres a hijos.

```
@Entity
@Table(name = "PROFESOR")
public class Profesor implements Serializable {


@Id
@GeneratedValue(strategy= GenerationType.IDENTITY)
@Column(name = "ID_PROFESOR")
private Long idProfesor;

@ManyToMany(cascade = {CascadeType.PERSIST,
                       CascadeType.MERGE})
@JoinTable(name = "PROFESOR_MODULO",
joinColumns = @JoinColumn(name = "ID_PROFESOR"),
inverseJoinColumns = @JoinColumn(name = "ID_MODULO"))
private Set<Modulo> modulos = new HashSet<>();
```

5. Anotación **@JoinTable** para establecer la tabla intermedia
6. **joinColumns** para establecer la relación entre la clave foránea de la tabla intermedia con nuestra clave primaria y **inverseJoinColumn** para establecer la relación de la clave foránea de la tabla intermedia con la clave de Modulo

Profesor: Entidad Propietaria

Es importante añadir métodos que permita añadir o eliminar elementos de la relación desde la parte propietaria.

```
//getters y setters
public Set<Modulo> getModulos() {
return modulos;
}

public void setModulos(Set<Modulo> modulos) {
this.modulos = modulos;
}

//métodos para facilitar la matriculación y desmatriculación de módulos
public boolean anadirModulo(Email email) {
email.setProfesor(this);
return getEmails().add(email);
}

public void eliminarModulo(Modulo modulo) {
this.modulos.remove(modulo);
modulo.getProfesores().remove(this);
}
```

Módulo: Entidad poseída

```
public class Modulo  implements Serializable {

private Long idModulo;
```

Módulo: Entidad poseída

1. Establecemos las anotaciones básicas.

```
@Entity
@Table(name = "MODULO")
public class Modulo  implements Serializable {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
@Column(name = "ID_MODULO")
private Long idModulo;
```

Módulo: Entidad poseída

1. Establecemos las anotaciones básicas.
2. Como un Modulo se relaciona con varios Profesores establecemos una variable profesores de tipo **Set**.
3. Anotamos los módulos con las anotación **@ManyToMany**

```
@Entity
@Table(name = "MODULO")
public class Modulo  implements Serializable {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
@Column(name = "ID_MODULO")
private Long idModulo;


private Set<Profesor> profesores = new HashSet<>();
```

Módulo: Entidad poseída

1. Establecemos las anotaciones básicas.
2. Como un Modulo se relaciona con varios Profesores establecemos una variable profesores de tipo **Set**.
3. Anotamos los profesores con las anotación **@ManyToMany**

```
@Entity
@Table(name = "MODULO")
public class Modulo  implements Serializable {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
@Column(name = "ID_MODULO")
private Long idModulo;

@ManyToMany(mappedBy = "modulos")
private Set<Profesor> profesores = new HashSet<>();
```

Módulo: Entidad poseída

1. Establecemos las anotaciones básicas.
2. Como un Modulo se relaciona con varios Profesores establecemos una variable profesores de tipo **Set**.
3. Anotamos los porfesores con las anotación **@ManyToMany**

```
@Entity
@Table(name = "MODULO")
public class Modulo  implements Serializable {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
@Column(name = "ID_MODULO")
private Long idModulo;

@ManyToMany(mappedBy = "modulos")
private Set<Profesor> profesores = new HashSet<>();
```

4. Empleamos el atributo **mappedby** para establecer el nombre del atributo en la entidad poseedora que mapea la relación. No es necesario establecer el @JoinTable ya que se establece del otro lado.

```
@ManyToMany(cascade = {CascadeType.PERSIST,
                       CascadeType.MERGE})
@JoinTable(name = "PROFESOR_MODULO",
joinColumns = @JoinColumn(name = "ID_PROFESOR"),
inverseJoinColumns = @JoinColumn(name = "ID_MODULO"))
private Set<Modulo> modulos = new HashSet<>();
```